

Authentication Modernization — Deep Research Findings (HelixCode)

Revision: 1 **Last modified:** 2026-07-01T18:26:19Z **Authority:** Operator standing “frequent deep research” directive (§11.4.8 / §11.4.99 / §11.4.150) **Scope:** Modernizing auth for a self-hosted, single-real-user web IDE — code-server behind a Caddy forward_auth gate, current auth = manual SSH-key challenge-response (ssh-keygen -Y sign → paste armored signature; no passwords stored). **Stack under consideration:** Caddy (reverse proxy + forward-auth) → Gin (Go) auth service → code-server (--auth none on 127.0.0.1:8080) under systemd-user + rootless-podman. **Honesty note (§11.4.6):** Every maturity/licence/behaviour claim below is cited in the “Sources verified” footer with an access date. Where a fact was NOT fully pinned down this pass, it is marked VERIFY: rather than asserted. This report is a research artefact — it recommends, it does not implement, and none of it substitutes runtime validation of a chosen path (§11.4.108 / §11.4.40).

0. TL;DR

Three things move the needle for THIS model (one real human, host-native, no multi-tenant IdP needed):

1. **Add WebAuthn/passkeys (FIDO2) to the existing Gin gate** via the go-webauthn/webauthn library — same hardware key the user already trusts for SSH, phishing-resistant, one-tap instead of paste-a-signature. Biggest UX+security win for the least architectural change.
2. **Close the 127.0.0.1:8080 --auth none local-bypass** by moving code-server onto a **unix domain socket inside a 0700 dir** (Caddy reverse-proxies to it), optionally belt-and-suspenders with a dedicated UID + iptables OUTPUT owner-match or a network namespace / gVisor loopback. This removes the “any local user/process can reach the IDE unauthenticated” hole entirely.
3. **(Bigger lever, different model) Consider a WireGuard / Headscale network layer** so the IDE is never publicly exposed at all — auth becomes device-key based at the tunnel. Strong

for a single user; trade-off is you lose “open it in a browser on any random machine”.

Everything else (Authelia/Authentik/oauth2-proxy/Pomerium/Teleport/Cloudflare Access, short-lived SSH certs, OAuth device flow) is analyzed below with fit, licence, maturity, and concrete integration notes — most are heavier than a single-real-user host-native model warrants, and two carry licence/ban-risk caveats worth flagging.

1. Passwordless / UX-improving auth mechanisms

1.1 WebAuthn / passkeys / FIDO2 (the recommended primitive)

How it works. FIDO2 = W3C **WebAuthn** (browser $\rightleftharpoons$ relying-party API) + FIDO **CTAP2** (browser $\rightleftharpoons$ authenticator). The server stores only a **public key**; the private key never leaves the authenticator (platform TPM/Secure-Enclave, or a roaming hardware key like a YubiKey). Login proves *possession of a private key* bound to the origin (RP ID), which makes it **phishing-resistant by construction** — a credential registered for `helixcode.example.com` cannot be replayed against a look-alike origin. WebAuthn is at **Level 3** (W3C candidate recommendation, March 2025). A “passkey” is just a discoverable FIDO2 credential (may be device-bound on a hardware key, or synced by an OS credential provider). Hardware keys have carried device-bound passkeys since 2019. NIST SP 800-63-4 (2025) explicitly pushes phishing-resistant authenticators like passkeys. [FIDO Alliance; W3C WebAuthn; Yubico; NIST]

Security trade-offs. - Strong: no server-stored shared secret to exfiltrate; phishing-resistant; MFA-in-one-gesture (possession + biometric/PIN user-verification). - Watch: **account-recovery / lost-authenticator** is the hard part — you **MUST** keep a second factor path. For a single real user that’s naturally the **existing SSH-key challenge**, which becomes the recovery/bootstrap path rather than the daily path. - Watch: WebAuthn requires **HTTPS** and a stable **RP ID** (your domain). Already satisfied by the Caddy TLS front.

Fit for single-real-user host-native. Excellent. You need exactly **one** stored credential for one user; no user directory, no IdP, no session broker. It slots directly into the Gin gate you already own.

Licence / maturity — the incorporable library. `go-webauthn/webauthn` (`github.com/go-webauthn/webauthn`) is **FIDO2-conformance-tested**, actively maintained, supports

passwordless / usernameless and **single relying party** partitioning by RP ID, and is the maintained successor to the older, now-superseded duo-labs/webauthn. It is a backend library (BSD-3-style OSS licence — VERIFY: exact SPDX in the repo LICENSE before shipping, \$11.4.99), storage-agnostic and framework-agnostic — a natural fit for Gin. Corbado's SDK survey lists it as the canonical Go WebAuthn backend. [go-webauthn README/ pkg.go.dev; Corbado survey]

Concrete integration path (Caddy + Gin + systemd-user + rootless-podman). - Add go-webauthn/webauthn to the Gin service. Configure RPID = <your FQDN>, RPOrigins = ["https://<your FQDN>"]. - Persist a **single** credential record (the user's public key + credential ID + sign-count) in whatever store the Gin gate already uses; registration is a one-time sudo/CLI-gated bootstrap (so nobody can self-enroll a new key over the wire). - Gin exposes /auth/webauthn/begin + /auth/webauthn/finish; on success it sets the same session cookie / returns the same 2xx that Caddy forward_auth already keys on (see §1.6) — so **the Caddy side does not change**. - Keep the SSH-key ssh-keygen -Y sign path as the **recovery** path (lost/broken key). - Because the authenticator can be the **same YubiKey** the user already uses for SSH, this is additive, not a second gadget to carry.

1.2 FIDO2 hardware security keys (as the authenticator behind §1.1)

Not a separate stack — it's the roaming-authenticator instance of §1.1. YubiKey-class keys give device-bound, non-exportable credentials with user-verification (PIN/touch). For a threat model where the operator wants "my laptop could be compromised but my login still can't be phished or replayed", a hardware key is the strongest single choice and composes with everything below. [Yubico FIDO2]

1.3 Short-lived CA-signed SSH certificates

How it works. An SSH CA signs a user's public key into a **short-TTL certificate** (e.g. minutes-hours) carrying principals/validity/extensions; the host trusts the CA (TrustedUserCAKeys) instead of a static authorized_keys. Rotation is automatic — expiry replaces revocation for most cases.

Options / licence / maturity. - **step-ca (Smallstep)** — **Apache-2.0**, single binary, has a **built-in SSH CA** (relatively unique), lightweight, "your own Let's Encrypt" model, can gate issuance behind an OIDC provider. Positioned as the cheapest/simplest to run, designed for *thousands not millions*. Best fit if you want SSH certs without Teleport/Vault weight. [Smallstep; Axelspire CA comparison] - **HashiCorp Vault SSH secrets**

engine — signs SSH public keys via a CA, policy-enforced. Best if you *already* run Vault; otherwise it’s a large dependency for one user. [HashiCorp]

Fit. Orthogonal improvement to the **SSH side** (server login), not the **browser IDE gate**. It makes your SSH story cert-based and auto-rotating, but it does not by itself improve the web-IDE login UX. Reasonable as a *later, SSH-hardening* item; not the primary lever here.

1.4 OAuth 2.0 / OIDC device authorization grant (RFC 8628)

How it works. Device shows a `user_code` + `verification_uri`; user approves in a browser on any device; the device polls the token endpoint until approved. Designed for input-constrained / headless clients. [RFC 8628 / oauth.net]

Fit for THIS model — weak. Device flow presupposes an **OAuth/OIDC authorization server** (an IdP) to approve against. For a single real user with no existing IdP, adding one (or depending on an external one) is a large, ban-risk-adjacent dependency for little gain — it re-introduces exactly the “external identity provider” the current host-native design avoids. Keep it in the back pocket ONLY if an IdP is adopted for other reasons.

1.5 Identity-aware proxies (the “replace/augment the Gin gate” class)

Product	What it is	Self-host	Licence (VERIFY current)	RAM/weight	Passkey/WebAuthn	Fit nat.
Authelia	Forward-auth SSO/MFA portal (not a token issuer)	Yes	Apache-2.0	~20-25 MB	Yes — passwordless passkeys, OIDC-certified	Bes Cad batt (TO limi bes)
Authentik	Full IdP (OIDC/OAuth2/SAML/LDAP, self-service)	Yes	Open-core (VERIFY: MIT core + gated enterprise)	2 cores + 2 GB min	Yes (incl. passkey-only flows)	Ove
oauth2-proxy	Thin forward-auth that	Yes	MIT	Small	Only via the upstream IdP	Nee the moc

Product	What it is	Self-host	Licence (VERIFY current)	RAM/weight	Passkey/WebAuthn	Fit nat.
Pomerium	delegates to an external OIDC/OAuth2 IdP Identity- & context-aware access proxy, fine-grained authz	Yes (core)	Apache-2.0 core (VERIFY:)	Medium	Via IdP / context	Hea grai app
Teleport	Access proxy for SSH/K8s/DB/web, certs, session recording	Yes	AGPLv3 OSS repo; Community Edition binaries commercial-licensed since v16	Heavy	Yes	Lic hea
Cloudflare Access	SaaS zero-trust gateway	No (SaaS)	Proprietary hosted	n/a	Via IdP	Not dep self- cave

Caveats to flag honestly (§11.4.99 / §11.4.112): - **Teleport licence** — the OSS core repo relicensed **Apache-2.0** → **AGPLv3** on 2023-12-01, and **Community Edition binaries switched to a commercial licence starting v16 (June 2024)**: free for personal/hobby use, but companies are restricted (≤ 100 employees / $< \$10\text{M}$ revenue; no resale/embed). Building from source stays AGPLv3. For a personal single-user IDE this is *usable* but the licence trajectory is a caveat; AGPL obligations attach if you distribute a modified service. [Teleport blog ×2; GitHub discussion #39158] - **Cloudflare Access** is a **hosted SaaS** — not open-source, not self-hostable, and puts a third party in your auth path. That directly contradicts the “self-hosted, no external dependency” premise; **do not adopt** unless the model changes. (Free tier exists but that is irrelevant to the architectural mismatch.) - **oauth2-proxy / Pomerium** both fundamentally expect an **external IdP** as the identity source. Pomerium is the stronger *authorization* engine (best-known oauth2-proxy alternative), but for a single user with one app, its fine-grained authz is unused weight.

If you ever want to retire the bespoke gate: Authelia is the one to pick — Apache-2.0, ~20-25 MB, native Caddy `forward_auth` at `/api/authz/forward-auth`, and native **passkey/WebAuthn passwordless** — so it delivers §1.1's UX without you writing/maintaining the crypto. The trade-off is it introduces a session/user store that overlaps your current Gin logic. Choosing between “add go-webauthn to Gin” (§1.1) and “adopt Authelia” is really “own the ~200 lines of WebAuthn glue” vs “operate one more service”. For one user, §1.1 keeps the surface smaller; Authelia wins if you also want TOTP + rate-limiting + a maintained login portal for free. [Authelia site + Caddy integration doc; Authelia-vs-Authentik comparisons]

1.6 How the Caddy `forward_auth` seam behaves (so any of the above drops in)

`forward_auth` proxies a **clone** of each request (GET, body not consumed) to your auth upstream; **2xx** ⇒ **allow** and `copy_headers` are copied onto the real request; **any other status** ⇒ **the upstream response is returned to the client** (typically a redirect to the login page). Caddy by default **strips fabricated identity headers** and trusts no proxy unless you set `trusted_proxies` — keep that tight. This means: whatever you put behind the gate (your Gin service, or Authelia) just has to answer **2xx/!2xx** on the verify endpoint, so §1.1 requires **zero** Caddy changes. Authelia's canonical form is `forward_auth authelia:9091 { uri /api/authz/forward-auth; copy_headers Remote-User Remote-Groups Remote-Email Remote-Name }`. [Caddy `forward_auth` docs; Authelia Caddy integration]

2. Keeping SSH-key-centric login, but smoother/safer

The current flow (`ssh-keygen -Y sign` → paste armored signature) is **cryptographically sound** and already passwordless with no stored secret — the pain is purely UX (manual copy/paste). Ways to keep the SSH-key center of gravity while smoothing it:

- **WebAuthn on the same hardware key (recommended, = §1.1).** A FIDO2 YubiKey can hold *both* an SSH key (`sk-ssh-ed25519`) *and* a WebAuthn credential. So “SSH-key-centric” and “passkey” need not be different hardware — the user taps the same key, but the browser handles the challenge instead of a terminal + clipboard. This is the smoothest safe upgrade and why §1.1 is ranked #1.
- **`ssh-keygen -Y` hygiene (harden what you already do).** OpenSSH signatures (8.0+) verify against an **allowed_signers** file (principals + keytype + key, `AUTHORIZED_KEYS`-like

format); the verifier must match the `-I` identity to a principals pattern and a **namespace** (`-n`). Best practice: use a **custom namespace** `helixcode-login@<your.domain>` (prevents cross-domain signature confusion / replay of a signature made for another purpose), a **dedicated signing key** separate from the daily auth key, and a passphrase/hardware-backed key. Confirm your gate pins BOTH the namespace and a server-issued, single-use, short-TTL challenge nonce (so a captured signature can't be replayed). [OpenBSD `ssh-keygen(1)`; agwa "sign arbitrary data"; Sigstore "SSH is the new GPG"]

- **ssh-agent / browser bridge (smoother, no new crypto).** Instead of shell-and-paste, a small helper (browser extension or localhost helper) can call the local ssh-agent to sign the server's challenge and POST it back — same trust model, no clipboard. This is a UX wrapper over the existing scheme; lower ceiling than §1.1 but zero change to the security primitive.
 - **Short-lived SSH certs (= §1.3)** harden the SSH-server side but don't smooth the *browser* login; treat as a separate SSH-hardening track.
-

3. Closing the code-server --auth none on 127.0.0.1:8080 local-bypass residual

The hole. A localhost TCP port with `--auth none` is reachable by **any local user or process** on the host (and any container sharing the host net namespace), completely bypassing the Caddy gate. On a single-user box the blast radius is small but real (a compromised unrelated process, a second UID, a mis-scoped container). The code-server maintainers themselves call a localhost TCP port "less secure on shared hosts" and recommend a unix socket. [code-server discussion #4524]

Ranked, composable mitigations:

1. **Unix domain socket + 0700 parent dir (primary, recommended).** Launch code-server with `--socket <path>` so it listens on a **unix socket** instead of TCP; Caddy reverse-proxies to `unix/<path>`. Access control becomes **filesystem permissions**. Note: code-server's socket has historically defaulted to **0755 (world-readable, owner-writable)**, and a request to set the socket mode/group (`--socket-mode`) was raised as issue #1466 — so **do not rely on the socket file mode alone**; place the socket inside a directory that is 0700 owned by the code-server UID (e.g. `/run/user/<uid>/helixcode/cs.sock`), which denies traversal to every other UID regardless of the socket's own mode. VERIFY: whether current code-server exposes `--socket-mode` via code-server `--help` before depending

on it (§11.4.99). [code-server guide/discussion; issue #1466; unix-socket-permissions references]

2. **Dedicated UID + iptables OUTPUT owner-match (defense-in-depth if a TCP port must remain).** A connect to 127.0.0.1:8080 from another local user emits **OUTPUT** packets *owned by that user's UID*; iptables -A OUTPUT -d 127.0.0.1 --dport 8080 -m owner ! --uid-owner <caddy_uid> -j REJECT blocks cross-UID loopback connects (owner match only applies to locally-generated packets, i.e. the OUTPUT chain — that's exactly this case). Run Caddy and code-server as distinct UIDs so the rule is meaningful. [Linux Journal "iptables for Local Security"]
3. **Network namespace isolation.** Put code-server in its own net namespace so its loopback is private; bridge only Caddy in. Rootless-podman already gives each pod a separate netns — so **running code-server inside a rootless-podman pod that does NOT publish 8080 to the host, and having Caddy reach it over the pod's socket/veth, closes the hole natively** (compose with #1: publish a unix socket into a shared 0700 volume rather than a host TCP port).
4. **gVisor (runc) for strong sandboxing of untrusted workloads.** gVisor's **netstack runs the loopback entirely inside the sandbox, isolated from the host** — so a code-server (or the code it executes) under gVisor cannot reach or be reached on the host loopback. Cost: ~10-30% overhead on I/O-heavy work (an IDE building/compiling will feel some). Best reserved for "I run untrusted code in this IDE" threat models rather than just closing the localhost port (#1+#3 already do that cheaply). [gVisor networking docs]

Recommended combination: #1 (unix socket in a 0700 dir) + #3 (don't publish the port from the rootless-podman pod) removes the residual with essentially no overhead. Add **#2** as a cheap belt-and-suspenders if a TCP listener has to exist. Reserve **#4** for the separate "sandbox untrusted code execution" goal.

Top recommendations for HelixCode

Ranked by (game-changing value × incorporability × low risk) for the single-real-user, host-native, Caddy+Gin+systemd-user+rootless-podman model:

#1 — Add WebAuthn/passkeys to the existing Gin gate (go-webauthn). *Biggest UX+security win, smallest architectural change, keeps the host-native no-IdP model, reuses the same hardware key as SSH, phishing-resistant.* **Next step:** vendor go-webauthn/webauthn into the Gin service; set RPID/RPOrigins to your FQDN; store ONE credential via a sudo/CLI-gated bootstrap registration; add /auth/webauthn/begin+/finish; on success emit

the same 2xx/session the Caddy forward_auth already keys on (no Caddy change); keep ssh-keygen -Y sign as the recovery path. VERIFY: the library's exact SPDX licence in its LICENSE before shipping (\$11.4.99).

#2 — Close the --auth none localhost residual with a unix socket + 0700 dir + unpublished pod port. *Eliminates the “any local process bypasses the gate” hole with ~zero overhead.* **Next step:** launch code-server with --socket /run/user/<uid>/helixcode/cs.sock in a 0700 owner-only dir; point Caddy at unix//run/user/<uid>/helixcode/cs.sock; ensure the rootless-podman pod does NOT publish 8080 to the host; optionally add the iptables OUTPUT --uid-owner belt-and-suspenders. VERIFY: current --socket/--socket-mode behaviour via code-server --help.

#3 — (Evaluate) network-layer access via WireGuard/Headscale, OR adopt Authelia if you want a batteries-included portal. - *WireGuard/Headscale (game-changer, different model):* stop exposing the IDE publicly at all — reach it over a self-hosted WireGuard mesh (Headscale = open-source, self-hosted Tailscale control-plane reimplementation; raw WireGuard = ~10-min single tunnel, ~0.5% CPU). Auth becomes device-key based; the public attack surface goes to zero. **Trade-off:** you lose “open it in a browser on an arbitrary/locked-down machine” — every client must run the tunnel. **Next step:** prototype a laptop↔host WireGuard tunnel and bind Caddy to the WG interface only; decide whether browser-from-anywhere is a hard requirement before committing. - *Authelia (if you'd rather not maintain WebAuthn glue):* Apache-2.0, ~20-25 MB, native Caddy forward_auth, native passkey/WebAuthn passwordless. **Next step:** stand up Authelia as the forward-auth upstream (uri /api/authz/forward-auth, copy_headers Remote-User ...), enroll one passkey, and retire the bespoke Gin auth logic — only worth it if you also want its TOTP + sessions + rate-limiting for free.

Explicitly NOT recommended for this model: Cloudflare Access (SaaS, third-party in the auth path — contradicts self-hosting); OAuth device flow / oauth2-proxy (both require an external IdP you don't have); Teleport (AGPL/commercial-CE licence caveat + far too heavy for one user); Authentik (a full IdP — overkill for a single account).

Sources verified 2026-07-01

Standards / primitives: - W3C WebAuthn + FIDO2 overview — FIDO Alliance passkeys: <https://fidoalliance.org/passkeys/> (accessed 2026-07-01) - Yubico FIDO2 passwordless: <https://www.yubico.com/authentication-standards/fido2/> (accessed 2026-07-01) - WebAuthn/FIDO2/passkey guide (Level 3, NIST SP 800-63-4 context): <https://terrazone.io/webauthn-complete-guide->

passwordless-fido2-passkeys/ (accessed 2026-07-01) - OAuth 2.0 Device Authorization Grant — RFC 8628: <https://datatracker.ietf.org/doc/html/rfc8628> (accessed 2026-07-01) - OAuth device flow overview: <https://oauth.net/2/device-flow/> (accessed 2026-07-01) - OpenSSH ssh-keygen(1) manual (-Y sign/verify, allowed_signers, namespaces): <https://man.openbsd.org/ssh-keygen.1> (accessed 2026-07-01) - “It’s Now Possible To Sign Arbitrary Data With Your SSH Keys” (agwa): https://www.agwa.name/blog/post/ssh_signatures (accessed 2026-07-01) - “SSH is the new GPG” (Sigstore blog): <https://blog.sigstore.dev/ssh-is-the-new-gpg-74b3c6cc51c0/> (accessed 2026-07-01)

Go WebAuthn library (incorporable): - go-webauthn/webauthn (FIDO2-conformant, passwordless, single RP): <https://github.com/go-webauthn/webauthn> (accessed 2026-07-01) - go-webauthn pkg.go.dev (RP ID partitioning, passwordless/usernameless): <https://pkg.go.dev/github.com/go-webauthn/webauthn/webauthn> (accessed 2026-07-01) - Passkey SDK/library survey (Corbado): <https://www.corbado.com/blog/best-passkey-sdks-libraries> (accessed 2026-07-01)

Identity-aware proxies: - Authelia (Apache-2.0, ~20-25 MB, OIDC-certified, passkeys): <https://www.authelia.com/> (accessed 2026-07-01) - Authelia Caddy integration (forward_auth / /api/authz/forward-auth / copy_headers): <https://www.authelia.com/integration/proxies/caddy/> (accessed 2026-07-01) - Caddy forward_auth directive (request clone, 2xx=allow, copy_headers, trusted_proxies): https://caddyserver.com/docs/caddyfile/directives/forward_auth (accessed 2026-07-01) - Authelia vs Authentik 2025/2026 (weights, capabilities): <https://blog.houseoffoss.com/post/authelia-vs-authentik-which-self-hosted-identity-provider-is-better-in-2025> and <https://www.cerbos.dev/blog/authelia-vs-authentik-2026-idp> (accessed 2026-07-01) - Pomerium (identity-aware proxy; oauth2-proxy alternative; delegates to IdP): <https://www.pomerium.com/blog/best-oauth2-proxy-alternative> and <https://github.com/pomerium/pomerium> (accessed 2026-07-01) - Teleport OSS → AGPLv3 relicense (2023-12-01): <https://goteleport.com/blog/teleport-oss-switches-to-agpl-v3/> (accessed 2026-07-01) - Teleport Community Edition → commercial licence from v16: <https://goteleport.com/blog/teleport-community-license/> and <https://github.com/gravitational/teleport/discussions/39158> (accessed 2026-07-01)

Short-lived SSH certificates: - Smallstep step-ca vs HashiCorp Vault SSH (Apache-2.0, built-in SSH CA, lightweight): <https://smallstep.com/hashicorp-vault-vs-smallstep-certificate-manager/> (accessed 2026-07-01) - Private CA comparison (step-ca / Vault PKI): <https://axelspire.com/vault/vendors/private-ca-comparison/> (accessed 2026-07-01) - HashiCorp Vault SSH-at-scale (SSH

secrets engine as CA): <https://www.hashicorp.com/en/blog/managing-ssh-access-at-scale-with-hashicorp-vault> (accessed 2026-07-01)

code-server local-bypass hardening: - code-server: allow access outside localhost / socket recommendation (#4524): <https://github.com/coder/code-server/discussions/4524> (accessed 2026-07-01) - code-server: set unix socket permissions (--socket-mode request, #1466): <https://github.com/coder/code-server/issues/1466> (accessed 2026-07-01) - code-server securely access & expose guide: <https://coder.com/docs/code-server/guide> (accessed 2026-07-01) - iptables owner-match for local security (OUTPUT chain, --uid-owner): <https://www.linuxjournal.com/article/6091> (accessed 2026-07-01) - gVisor networking (netstack loopback isolated in sandbox): https://gvisor.dev/docs/user_guide/networking/ (accessed 2026-07-01)

Network-layer alternative: - Tailscale vs WireGuard vs Headscale (self-hosted control plane, no exposed ports): <https://netguardia.com/privacy/self-hosting/tailscale-vs-wireguard-vs-headscale-remote-access-without-exposing-ports/> (accessed 2026-07-01) - Tailscale/WireGuard model + control-plane auth: <https://tailscale.com/kb/1035/wireguard> (accessed 2026-07-01)

Negative findings / gaps (§11.4.99(B)): exact current SPDX licence of go-webauthn/webauthn, the precise licence terms of Authentik's enterprise tier, and whether current code-server ships a --socket-mode flag were NOT fully pinned this pass — each is tagged VERIFY: above and must be confirmed against the primary source before any implementation commit. This report is research only and is not a substitute for runtime validation of a chosen path (§11.4.108 / §11.4.40).